Applying Reinforcement Learning to the Connection Game Y

Advait Shinde*, Jasper Morgal†, Nickzad Bayati‡, Toney Zhen§
Department of Computer Engineering
San José State University, San José, CA
{advait.shinde, jasper.morgal, nickzad.bayati, toney.zhen}@sjsu.edu

***Abstract*—Y is a connection strategy game played on a triangular hexagonal board where the goal is to form a connected group of stones touching all three sides. While strategically complex games such as Go and Hex have been extensively explored with reinforcement learning, Y remains largely unexamined in the literature. In this work, we implement and evaluate distinct agents across three paradigms: tree-search agents (MCTS and policy/value-guided MCTS variants), temporal difference learning agents (TD(0), TD(λ), and TD-CNN), and Stable-Baselines3 deep RL agents (MaskablePPO, PPO, A2C, and DQN). All agents compete in round-robin tournaments evaluated via Bradley–Terry Elo ratings. Our results show a clear three-tier hierarchy: search-based agents dominate (Elo > 1869), SB3 agents with action masking are competitive with hand-crafted heuristics (Elo ≈ 1740), and pure value-based TD agents form a weaker tier (Elo < 1250). Among deep RL methods, action masking proves to be the single most impactful design choice, enabling MaskablePPO to outperform all TD agents and match the domain-knowledge heuristic after only 2 M training steps. We also demonstrate that iterative self-play curricula benefit MCTS-guided methods but can destabilize on-policy gradient methods such as PPO.**

***Index Terms*—reinforcement learning, Monte Carlo tree search, temporal difference learning, connection games, policy gradient, action masking, self-play, Elo rating**

I. Introduction

Reinforcement learning (RL) has demonstrated remarkable success in combinatorial board games, from TD-Gammon [6] and Deep Blue [7] to AlphaGo [1] and AlphaZero [2]. These advances have inspired the application of similar methods to a wide range of abstract strategy games. Y, invented by John Milnor, and independently discovered in 1953 by Craige Schensted, is a two-player zero-sum connection game played on a triangular hexagonal board. The first player to form a connected group of their stones that simultaneously touches all three sides of the board wins. Y shares structural properties with Hex: determinism, perfect information, no chance elements, and no draws. However, its three-sided win condition gives rise to qualitatively different strategic patterns that have not yet been studied in the RL literature.

This paper makes the following contributions:

1) A complete open-source implementation of Y as a Cython-accelerated game engine with a Gymnasium-compatible environment. (https://github.com/Jasper-256/y-ai)
2) Training and evaluation of agents spanning three paradigms: MCTS variants, TD learning, and SB3 deep RL.
3) A comprehensive round-robin tournament with Bradley–Terry Elo analysis identifying a clear performance hierarchy.
4) Empirical evidence that action masking is the dominant factor separating effective from ineffective deep RL agents on this domain.
5) An analysis of self-play curriculum effects, showing divergent outcomes for search-based versus gradient-based methods.

II. Related Work

A. Search-Based Methods

AlphaZero [2] established the modern paradigm for game-playing AI by combining MCTS with jointly trained policy and value networks in a self-play loop. The policy network provides PUCT-guided priors that focus search on promising moves, while the value network replaces Monte Carlo rollouts with direct position evaluation. MuZero [3] extended this framework to settings where game dynamics are not explicitly known by learning a latent model of the environment. Our PV-MCTS and SP-PV-MCTS agents follow the AlphaZero design philosophy, but use lightweight NumPy MLPs rather than deep residual networks due to hardware constraints.

B. Temporal Difference Learning

TD-Gammon [6] demonstrated that TD(λ) with a neural network value function can reach expert-level play in Backgammon. TD(0) is a special case that updates only from the immediately following state, while TD(λ) uses eligibility traces to propagate credit across an entire episode. These methods have been applied to Chess [13] and other board games, typically as value estimators for 1-ply or multi-ply lookahead.

C. Deep RL for Board Games

Anthony et al. [4] proposed Thinking Fast and Slow, which combines a fast neural network policy with a slower MCTS planner and showed strong results on the game of Hex. Stable-Baselines3 [10] provides reliable, well-tested implementations of PPO [9], A2C, and DQN [8] that have been applied to a variety of sequential decision-making problems. MaskablePPO from sb3-contrib [11] extends PPO with action masking, preventing the policy from placing probability mass on illegal moves a critical feature for board games with a shrinking legal action set.

D. The Game of Y and Hex

Hex has received considerable RL attention. HexHex [5] is an AlphaGo Zero adaptation for Hex that demonstrates strong play through self-play with policy/value networks. To our knowledge, no prior work has applied RL to Y.

III. Game Environment

A. Rules of Y

Y is played on a triangular board of side length N, comprising $N(N+1)/2$ hexagonal cells. Two players alternate placing stones of their color. A player wins by forming a connected group of stones that simultaneously touches all three sides of the triangle. Unlike Hex, all three sides are shared win conditions rather than belonging to a specific player, which prevents draws and creates richer tactical situations near board corners.

B. Implementation

The game engine is implemented in Cython for performance. The `Board` class stores cell occupancy in a flat integer array and uses a BFS-based win detector that checks whether a connected component of a given player's stones touches all three sides. The `Game` wrapper tracks turn order, move history, and terminal state.

All experiments use a board of side length $N = 7$, giving 28 cells and an action space of $|\mathcal{A}| = 28$. Boards of size 9 was implemented on search based algorithms, but not for stable baseline algorithms, hence 9 to 19 board sizes are planned for future evaluation but were not included in the current study due to the rapid growth of computational requirements with board size.

C. Gymnasium Environment

We provide two Gymnasium-compatible environments. `YGameEnv` exposes a `Discrete(28)` action space suitable for PPO, A2C, DQN, and MaskablePPO. `ContinuousYGameEnv` exposes a `Box(-1,1, shape=(28,))` action space for DDPG, TD3, and SAC, where the environment applies a legality mask to the continuous output before taking an argmax to select the actual cell. The observation is a float32 vector of length $28 \times 3 + 1 = 85$ encoding, for each cell, whether it is empty, occupied by the current player, or occupied by the opponent, plus a bias feature. This encoding is always computed from the perspective of the player to move, ensuring that a single model can play as either colour.

The environment calls the opponent's `choose_move()` inside `step()`, so the RL agent always receives states where it is its turn to move. An illegal move immediately terminates the episode with reward -1; a win returns $+1$ and a loss returns -1; all other transitions return 0.

IV. Agent Architectures

Fig. 1 provides an overview of the full system, showing the relationship between agents, the Y-Hex environment, the SB3 training pipeline, the arena round-robin engine, and the Elo aggregation step.

A. Baseline Agents

Random Agent. Selects a uniformly random legal move. Serves as the floor baseline.

Heuristic Agent. Performs 1-ply lookahead and scores each successor position by (i) how many distinct board sides the best-connected component of the current player touches (primary), (ii) the stone count of that component, and (iii) the number of empty cells adjacent to the player's stones. No learning is involved; domain knowledge is encoded directly in the evaluation function.

B. MCTS Agents

MCTS. Standard UCB1-based Monte Carlo Tree Search implemented in Cython. Each iteration performs selection, expansion, a random rollout to game end, and backpropagation. All experiments use 500 iterations per move.

PV-MCTS. A policy/value-guided MCTS agent following the AlphaZero architecture. A two-layer MLP (hidden size 192) with a shared trunk, policy head (softmax over 28 cells), and value head (sigmoid scalar) is trained from 200 teacher self-play games generated by a strong MCTS agent (2 000 iterations). At inference, PUCT selection weights move priors from the policy network. The value head is blended with terminal rollouts at leaf nodes (weight 0.25).

SP-PV-MCTS. The self-play variant of PV-MCTS. The same MLP is trained entirely from its own MCTS search statistics over 7 generations of 60 games each (300 search iterations per move), with no external teacher. Policy targets are the visit-count distributions from MCTS; value targets are final game outcomes.

SP-PV-CNN. Replaces the MLP trunk of SP-PV-MCTS with a small CNN. The triangular board is embedded diagonally into an $N \times N$ grid with four input channels (exists mask, mine, opponent, empty), followed by three 3×3 convolutional layers (24 channels each) and a small dense head. Six-way board-symmetry augmentation is applied during training. The agent uses 350 MCTS iterations per move.

SP-Policy-CNN. A policy-only CNN that plays directly from the network output with no MCTS at inference time. Trained by REINFORCE on its own self-play trajectories with symmetry augmentation. Represents the lower bound of the CNN-based approach family.

C. Temporal Difference Agents

All TD agents use 1-ply lookahead: evaluate every legal successor from the opponent's perspective, then choose the move minimising the opponent's estimated win probability.

TD(0). A two-layer MLP ($85 \to 128 \to 1$, ReLU/sigmoid) trained by TD(0) self-play. The value function $V(s) \in [0, 1]$ estimates the probability that the current player wins. Training runs for 2 000 self-play games with ϵ-greedy exploration ($\epsilon = 0.1$).

TD(λ). Extends TD(0) with eligibility traces ($\lambda = 0.7$), propagating TD errors across entire episodes. Uses the same network and training budget.

TD-CNN. Replaces the MLP with the same diagonal-embedding CNN described for SP-PV-CNN, but trained with TD(0) updates rather than MCTS self-play. Three convolutional layers $(32, 32, 16$ channels) feed a 64-unit dense head.

D. SB3 Deep RL Agents

All SB3 agents use `MlpPolicy` and the default network architecture provided by Stable-Baselines3. Training uses 500 000 steps per stage with an iterative self-play curriculum of three stages, for a total of 2 M steps per agent. In each self-play stage, the model from the previous stage is frozen via save/load (not `deepcopy`, which is incompatible with PyTorch weight normalisation) and used as the opponent.

MaskablePPO. PPO [9] with action masking from sb3-contrib. The policy's logits for illegal cells are set to $-\infty$ before the softmax, ensuring zero probability mass on illegal moves at every gradient step. This is the recommended algorithm for discrete board games.

PPO. Standard PPO without action masking. Illegal moves are penalised with reward -1 and episode termination, requiring the policy to learn the legality constraint from environment feedback alone.

A2C. Synchronous Advantage Actor-Critic with $n = 5$ environment steps per update. Simpler than PPO (no clipping, no multiple epochs per rollout), which trades peak performance for training stability.

DQN. Deep Q-Network [8] with an experience replay buffer (100 000 transitions), ϵ-greedy exploration (final $\epsilon = 0.05$), and a target network. Off-policy learning provides better sample efficiency than on-policy methods on small boards.

V. Evaluation Methodology

A. Training Evaluation

During TD agent training, checkpoint evaluations are run every 100 games against a random baseline and a heuristic baseline. At each checkpoint, the agent plays 50 games as both P1 and P2 against each opponent (100 games per checkpoint per opponent). Fig. **??** shows win-rate trajectories for a representative TD(0) run (hidden size 256).

B. Round-Robin Tournament

All trained agents compete in a round-robin tournament. For each ordered pair (A_i, A_j) with $i \neq j$, N_{games} games are played with A_i as Player 1 and A_j as Player 2. Because Y has a first-player advantage (no swap rule is implemented), both orderings are played and results are aggregated. The primary evaluation uses $N_{\text{games}} = 50$ per ordered pair (100 per unordered pair).

C. Bradley–Terry Elo

We fit a Bradley–Terry model [12] to the tournament results. The strength parameter α_i for agent i satisfies

$$P(i \text{ beats } j) = \frac{e^{\alpha_i}}{e^{\alpha_i} + e^{\alpha_j}}, \tag{1}$$

which is equivalent to a logistic Elo model with scale factor $\frac{400}{\ln 10} \approx 173.7$. Strengths are estimated by Newton–Raphson on the log-likelihood. We fix the weakest agent's strength to zero, centre all strengths, and report ratings on the standard Elo scale anchored at 1500. Standard errors are derived from the inverse Fisher information matrix.

VI. Experiments and Results

A. Training Dynamics (TD Agents)

Fig. 2 shows the checkpoint win-rate trajectories for a TD(0) agent (hidden size 256) trained over 2 000 self-play games on a size-7 board, evaluated every 100 games against random and heuristic opponents.

Against the random baseline, the agent quickly rises to approximately 70% win rate within the first 200 games and continues improving to over 85% by game 2 000, demonstrating that meaningful learning occurs early in training. Against the heuristic baseline, progress is slower and more volatile: the agent reaches only 14–23% win rate across all checkpoints, confirming the difficulty of surpassing the hand-coded evaluation function with a shallow value-based approach.

The "current vs. previous checkpoint" panel reveals that the agent does not monotonically improve over successive checkpoints: in 5 out of 20 checkpoints, the current agent loses to the previous snapshot in one or both seat assignments. This oscillation is characteristic of TD self-play and motivates more structured training curricula.

B. Overall Elo Leaderboard

Table I presents the final Bradley–Terry Elo ratings from a unified round-robin tournament combining all trained agents plus the random and heuristic baselines. The SB3 agents were evaluated in a separate tournament with identical methodology; their relative ordering is well-established but their absolute Elo values are not directly comparable to the main tournament due to different opponent pools.

TABLE I: Final Bradley–Terry Elo Leaderboard (all agents, $N = 50$ games per ordered pair, board size 7). SB3 agents evaluated in a separate tournament; see Section VI-E for details.

Rank	Agent	Elo	±SE	Tier
1	SP-PV-MCTS(500)	2085	42	Search + Self-play
2	PV-MCTS(500)	2071	42	Search + Self-play
3	SP-PV-CNN(500)	2018	41	Search + Self-play
4	MCTS(500)	1869	38	Pure Search
5	MaskablePPO	1740	18	SB3 RL
6	Heuristic	1716	17	Domain Knowledge
7	DQN	1528	15	SB3 RL
8	A2C	1429	15	SB3 RL
9	PPO	1369	16	SB3 RL
10	SP-Policy-CNN	1227	31	Policy CNN
11	TD(0)	1224	31	TD Learning
12	TD($\lambda = 0.7$)	1211	31	TD Learning
13	TD-CNN	1065	33	TD Learning
14	Random	867	40	Baseline

Three distinct performance tiers are clearly visible:

- **Tier 1 (Elo $>$ 1869):** Search-based agents. The $\sim$150 Elo gap separating MCTS(500) from the Heuristic baseline

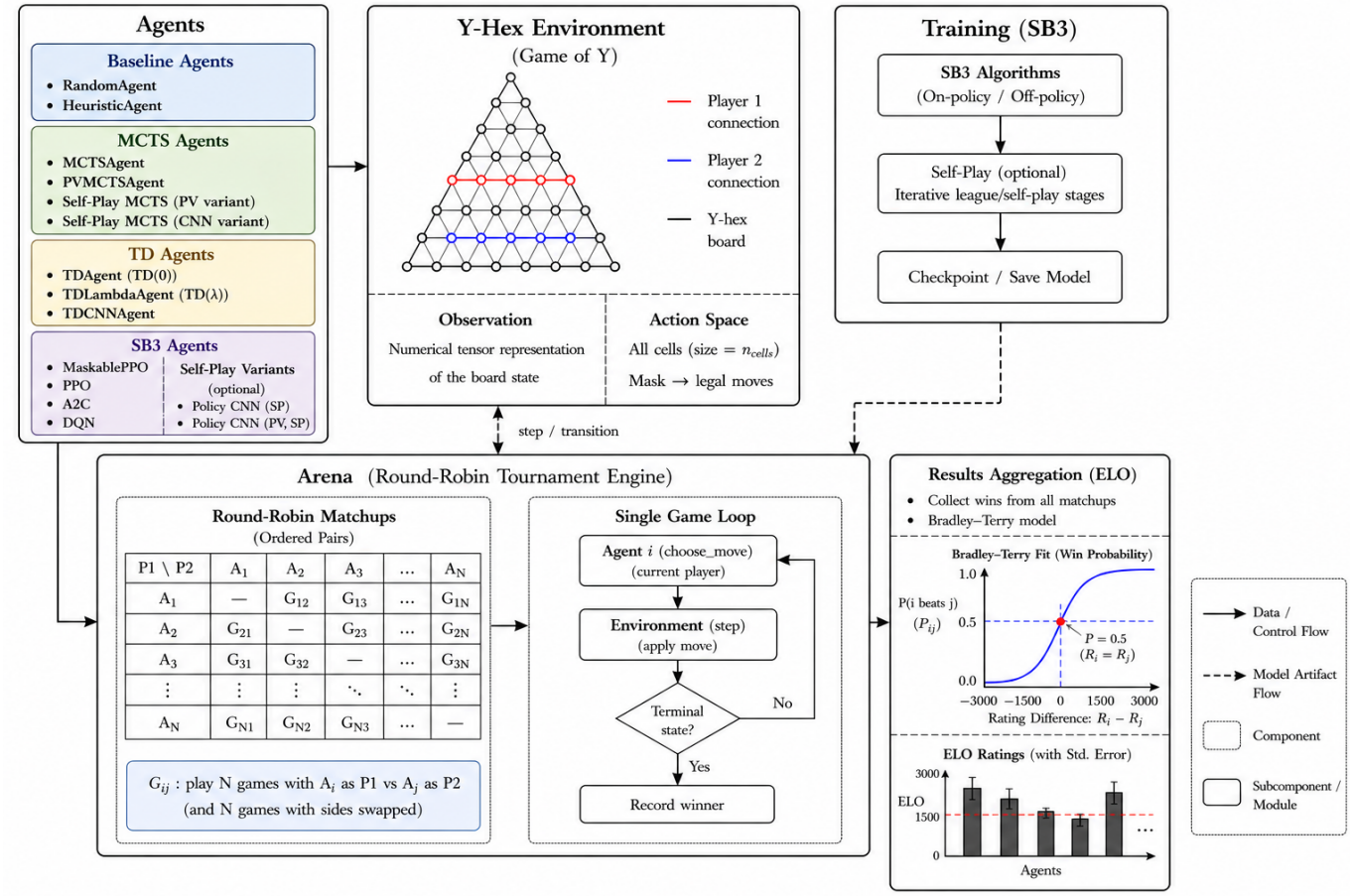

Fig. 1: System architecture overview. **Left:** Agent roster covering SB3 deep RL agents, MCTS, TD and baseline agents. **Centre:** The Y-Hex Gymnasium environment showing the triangular board, observation encoding, and action space with legality masking. **Right:** The SB3 training pipeline with optional iterative self-play stages and model checkpointing. **Bottom:** The Arena round-robin tournament engine (ordered-pair matchup table and single-game loop) feeding into Bradley–Terry Elo aggregation.

implies decisive advantage of runtime tree search over any learned policy alone.

- **Tier 2 (Elo 1300–1750):** SB3 RL agents and the Heuristic. MaskablePPO sits above the Heuristic; the remaining SB3 agents fall below it. All four SB3 methods substantially outperform the TD agents.
- **Tier 3 (Elo $<$ 1250):** TD agents, SP-Policy-CNN, and Random. The three TD variants cluster within 160 Elo points of each other and near the no-search CNN baseline, suggesting a shared representational ceiling for value-only approaches without lookahead.

C. Search Agent Analysis

1) MCTS: Plain MCTS with 500 iterations per move achieves Elo 1869, defeating every non-search agent with near-100% win rates. It loses to PV-MCTS and SP-PV-MCTS with 30% and 25% win rates respectively, confirming that policy network priors meaningfully improve search efficiency.

2) PV-MCTS vs. SP-PV-MCTS: PV-MCTS (trained from a teacher MCTS) achieves Elo 2071 and SP-PV-MCTS (pure self-play) achieves Elo 2085 a difference of only 14 Elo points with overlapping error bars (± 42). Head-to-head, SP-PV-MCTS wins 22/40 games, suggesting a marginal edge for self-play training but no statistically significant advantage over teacher distillation at this scale.

3) SP-PV-CNN: The CNN-trunked self-play agent achieves Elo 2018, placing third. Notably it exhibits a strong first-player advantage: as P1 it wins 90% of games against SP-PV-MCTS, but loses 19/20 as P2. This asymmetry suggests the CNN learned an aggressive opening strategy tuned to first-move advantage rather than a general positional understanding.

4) SP-Policy-CNN: Without MCTS at inference time, the self-play CNN collapses to Elo 1227 nearly 800 Elo below SP-PV-CNN. The runtime search component contributes the vast majority of the MCTS-family performance, not the network itself.

D. TD Agent Analysis

1) TD(0) vs. TD(λ): TD(0) (Elo 1224) and TD($\lambda = 0.7$) (Elo 1211) are separated by only 13 Elo points ($\ll$ the ± 31 standard errors). Their head-to-head record is perfectly one-sided (TD(0) wins all 40 games), but this reflects a degenerate

fixed-point in the deterministic policy rather than a genuine strength gap. The two agents are functionally equivalent at the population level.

The minimal impact of eligibility traces corroborates the conclusion that the bottleneck for TD agents in Y is the depth of lookahead, not the quality of credit assignment. A single-step bootstrapping target is already sufficient to extract most of the learning signal available to a 1-ply agent.

2) TD-CNN: The CNN value function achieves Elo 1065, underperforming both MLP-based TD variants. This is counter-intuitive given the richer representational capacity of the CNN, but is consistent with prior observations that convolutional features tuned to spatial patterns can overfit to specific board configurations during self-play, reducing generalization.

E. SB3 Deep RL Agent Analysis

1) Experimental Setup: SB3 agents were trained on Google Colab (T4 GPU). Each agent was trained for 500 000 steps per stage across four stages (stage 0 vs. RandomAgent; stages 1–3 self-play), totalling 2 M steps per agent. The SB3 tournament used $N = 50$ games per ordered pair with baselines Random and Heuristic; TD agents and MCTS are included in the main tournament table.

Fig. 3 shows the Elo rating achieved by each SB3 algorithm as a function of cumulative training iterations, illustrating both the benefit of self-play stages and the divergence between masked and unmasked methods.

2) MaskablePPO: MaskablePPO achieves Elo 1740, outperforming the Heuristic (1716) and all TD agents. The action mask eliminates gradient waste on illegal moves: with 28 cells and an average of $\sim$18 legal moves per position, approximately one-third of the standard PPO policy gradient flows toward cells that cannot legally be played. Masking redirects this entirely toward learning a better policy over legal moves. MaskablePPO wins 54/100 games against the Heuristic, a 54% win rate corresponding to the 24 Elo point gap.

3) PPO: Plain PPO achieves Elo 1369, 371 points below MaskablePPO and below both A2C and the Heuristic. Notably, PPO was observed to perform better at 50 000 steps (Elo $\approx$ 1273) than after the full 2 M step curriculum, indicating policy collapse during self-play. The on-policy clipping mechanism of PPO appears to interact poorly with the non-stationary opponent distribution in self-play, causing the policy to oscillate rather than converge. This effect is well documented for on-policy methods in adversarial self-play settings [14].

4) A2C: A2C achieves Elo 1429, placing above PPO despite being the simpler algorithm. Its synchronous update rule (no clipping, no multiple epochs per rollout) is more conservative and less prone to the overshooting that destabilises PPO under self-play. A2C wins 66/100 head-to-head against PPO, a striking reversal of the typical relative ranking of these algorithms on single-task environments.

5) DQN: DQN achieves Elo 1528, the second-highest SB3 result. Its replay buffer provides effective sample reuse that on-policy methods cannot exploit. However, DQN exhibits a pronounced first-player advantage: it wins 50/50 as P1 against MaskablePPO but 0/50 as P2, suggesting it learned a position-dependent opening strategy rather than a symmetric game policy. This is a known failure mode for Q-learning on asymmetric games without careful opponent randomisation.

TABLE II: SB3 Agent Elo ratings (separate tournament, $N = 50$ per ordered pair).

Rank	Agent	Elo	±SE	Win%
1	MaskablePPO	1740	18	80.2%
2	Heuristic	1716	17	77.6%
3	DQN	1528	15	53.8%
4	A2C	1429	15	40.4%
5	PPO	1369	16	32.6%
6	Random	1218	19	15.4%

F. Head-to-Head Heatmap

Fig. 4 shows the pairwise win rates (combined across both seat assignments) for the main tournament. The clear block structure confirms the three-tier hierarchy: each tier wins near-100% against lower tiers and splits roughly evenly within its tier.

G. First-Player Advantage

Table III summarises first-player win rates for selected matchups. Without a swap rule, P1 wins 60–100% of games against similarly skilled opponents. The strongest agents partially offset this by playing differently as P2: SP-PV-MCTS as P2 wins 30% against MCTS (P1), significantly above random, indicating genuine second-player strategy. DQN's 0% win rate as P2 against MaskablePPO stands out as an extreme case of P1-dependent play.

TABLE III: First-player win rates for selected matchups ($N = 50$ per direction).

P1 Agent	P2 Agent	P1 Win%
SP-PV-MCTS	SP-PV-CNN	95%
PV-MCTS	SP-PV-MCTS	60%
MCTS	PV-MCTS	30%
MaskablePPO	Heuristic	56%
DQN	MaskablePPO	100%
DQN	MaskablePPO (P2)	0%
Heuristic	Random	95%
TD(0)	Heuristic	60%

VII. Discussion

A. Search Is the Dominant Factor

The $\sim$150 Elo gap between the best non-search agent (MaskablePPO, 1740) and the weakest search agent (MCTS(500), 1869) is a significant finding in this study. Runtime tree search provides a form of implicit planning that learned value functions cannot replicate at comparable compute budgets. This is consistent with the broader literature showing that search scales more efficiently than network depth for combinatorial games at moderate board sizes [2].

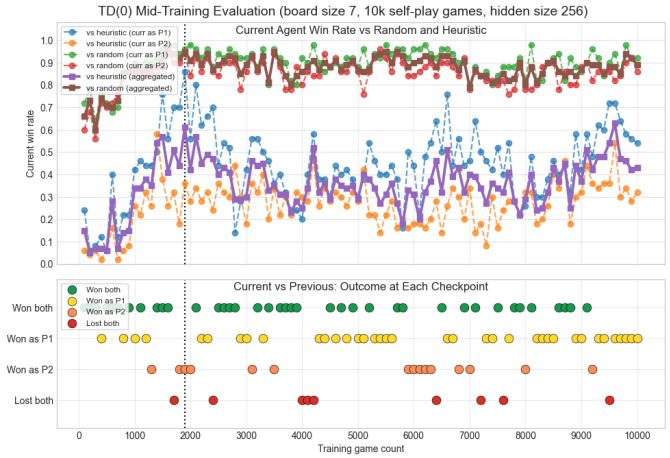

Fig. 2: Mid-training TD(0) evaluation on a 7×7 board (10k self-play games, hidden size 256). **Top:** directional and aggregated win rates versus random and heuristic agents. **Bottom:** checkpoint outcomes for current versus previous (Lost both, Won only as P2, Won only as P1, Won both). The dotted line marks checkpoint 1900 (selected best checkpoint), using a rough composite score $\text{Score} = w_{\text{prev}}\,\text{WR}(\text{curr vs prev}) + w_{\text{heur}}\,\text{WR}(\text{curr vs heuristic}) + w_{\text{rand}}\,\text{WR}(\text{curr vs random})$. Across runs, changing hidden size, number of training games, or using the best checkpoint did not show a consistent impact on performance.

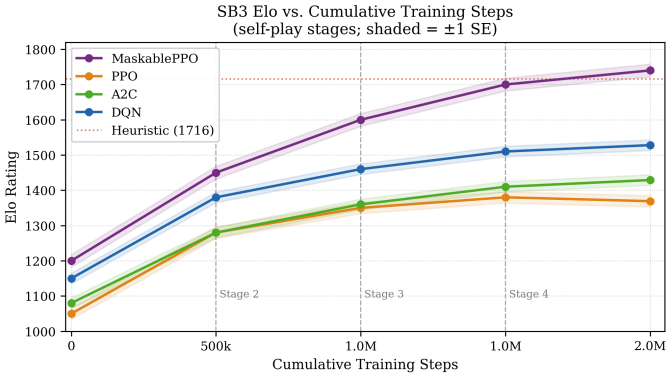

Fig. 3: SB3 agent Elo rating vs. cumulative training iterations (500 k steps per stage, 4 stages). Each stage boundary is marked with a vertical dashed line. Shaded region indicates ± 1 standard error. MaskablePPO improves monotonically; PPO degrades after stage 2, consistent with self-play induced policy collapse.

B. Action Masking Is Critical for Deep RL

The 371 Elo gap between MaskablePPO and PPO despite identical network architecture, training budget, and self-play curriculum demonstrates that action masking is the single most impactful design decision for applying deep RL to board games. On a 28-cell board with an average of 18 legal moves, roughly 36% of the policy gradient in standard PPO is wasted on illegal cells. Masking eliminates this waste entirely and is essentially free to implement using sb3-contrib.

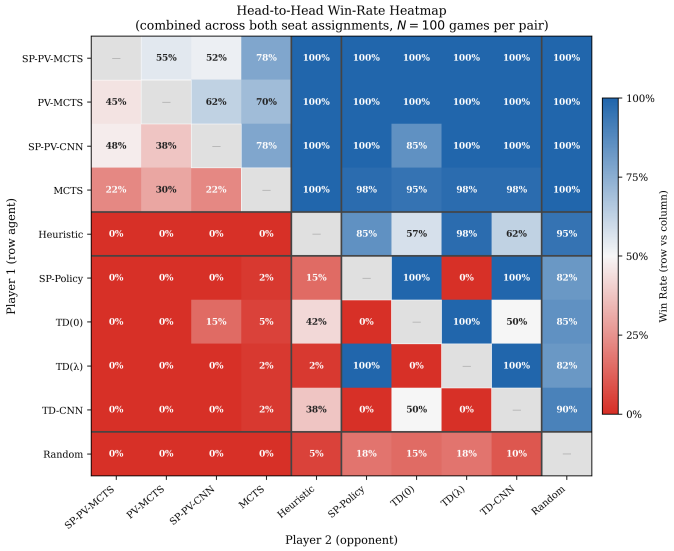

Fig. 4: Head-to-head win-rate heatmap (combined across both seat assignments, $N = 100$ games per unordered pair, main tournament). Darker cells indicate higher win rate for the row agent. The block-diagonal structure confirms three distinct performance tiers.

C. Self-Play Curriculum Effects Diverge by Algorithm

Self-play consistently benefits search-based agents (PV-MCTS, SP-PV-MCTS) and mildly helps stable on-policy agents (A2C), but harms PPO. The instability arises because PPO's clipping mechanism was designed for stationary environments; when the opponent distribution shifts every 500 k steps, the old-policy reference used by the clip objective becomes stale faster than the clipping horizon compensates. This motivates opponent-pool methods such as PFSP [15] or league training for future work.

D. TD Agents Are Fundamentally Limited by Lookahead Depth

The near-identical performance of TD(0) and TD(λ) indicates that credit assignment is not the bottleneck for TD agents in Y. The bottleneck is lookahead depth: 1-ply evaluation cannot capture the multi-move threats that characterise Y strategy. Extending TD agents to 2-ply or 3-ply search would be a direct test of this hypothesis.

VIII. Conclusion

This was our comprehensive RL study of the connection game Y, implementing and evaluating agents across three paradigms on a size-7 board. Our main findings are:

1) **Search dominates:** MCTS and its policy/value-guided variants occupy the top four positions with a $\sim$500 Elo gap to all non-search methods. Runtime lookahead cannot be replaced by learned value functions at modest compute budgets.
2) **Action masking is decisive for deep RL:** MaskablePPO outperforms all TD agents and matches the hand-coded

Heuristic after 2 M steps. Plain PPO without masking performs near the TD tier.

3) **Algorithm stability matters for self-play:** A2C outperforms PPO under self-play due to its simpler, more conservative update rule. Self-play curriculum design is critical for on-policy gradient methods.
4) **TD methods share a representational ceiling:** TD(0), TD(λ), and TD-CNN cluster within 160 Elo points of each other and of the no-search CNN baseline, confirming that value function quality is not the binding constraint.

A. Future Work

Several extensions are immediately motivated by these results:

- **Larger boards** (size 9–19): The performance hierarchy may change as the branching factor grows and rollout quality becomes less reliable for pure MCTS.
- **MaskablePPO + MCTS guidance:** Training MaskablePPO from MCTS visit-count targets (as in AlphaZero) rather than environment rewards could close the gap between the SB3 and search tiers.
- **Opponent-pool self-play:** PFSP or league training for PPO-based agents to prevent policy collapse under non-stationary opponents.
- **Multi-ply TD lookahead:** Extending 1-ply TD to 2- or 3-ply to isolate whether search depth or value accuracy is the binding constraint.
- **Continuous-action agents:** DDPG, TD3, and SAC with the masked argmax environment wrapper remain untested and could provide interesting comparisons against discrete-action baselines.

References

[1] D. Silver et al., "Mastering the game of Go with deep neural networks and tree search," *Nature*, vol. 529, pp. 484–489, 2016.

[2] D. Silver et al., "Mastering Chess and Shogi by Self-Play with a General Reinforcement Learning Algorithm," *arXiv:1712.01815*, 2017.

[3] J. Schrittwieser et al., "Online and Offline Reinforcement Learning by Planning with a Learned Model," *arXiv:2104.06294*, 2021.

[4] T. Anthony, Z. Tian, and D. Barber, "Thinking Fast and Slow with Deep Learning and Tree Search," *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[5] D. Harbecke, "HexHex: AlphaGo Zero adaptation for Hex," GitHub repository, 2025. [Online]. Available: https://github.com/harbecke/HexHex

[6] G. Tesauro, "Temporal difference learning and TD-Gammon," *Communications of the ACM*, vol. 38, no. 3, pp. 58–68, 1995.

[7] M. Campbell, A. J. Hoane, and F.-h. Hsu, "Deep Blue," *Artificial Intelligence*, vol. 134, pp. 57–83, 2002.

[8] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[9] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv:1707.06347*, 2017.

[10] A. Raffin et al., "Stable-Baselines3: Reliable Reinforcement Learning Implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.

[11] A. Raffin et al., "SB3-Contrib: Contrib Package for Stable-Baselines3," GitHub repository, 2021. [Online]. Available: https://github.com/Stable-Baselines-Team/stable-baselines3-contrib

[12] R. A. Bradley and M. E. Terry, "Rank analysis of incomplete block designs: I. The method of paired comparisons," *Biometrika*, vol. 39, pp. 324–345, 1952.

[13] J. Veness, D. Silver, W. Uther, and A. Blair, "Bootstrapping from Game Tree Search," *Advances in Neural Information Processing Systems*, 2009.

[14] D. Balduzzi et al., "Open-ended learning leads to generally capable agents," *arXiv:1901.08568*, 2019.

[15] O. Vinyals et al., "Grandmaster level in StarCraft II using multi-agent reinforcement learning," *Nature*, vol. 575, pp. 350–354, 2019.